32. Capabilities, seccomp, and least privilege (intro)

32. Capabilities, seccomp, and least privilege (intro)

Phase: Phase 9 — Security and SSH
Estimated time: 1–2 days
Slug: `32-capabilities-seccomp`

Learning objectives

- Explain Linux capabilities vs all-powerful root
- Know seccomp exists for syscall filtering
- Apply least privilege to services
- Relate to container security defaults

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Running everything as root is the oldest mistake in Linux. The kernel splits root's power into about 40 **capabilities** so a process gets only the powers it needs, and **seccomp** filters which system calls it may make at all. Container runtimes use both. The single most dangerous container shortcut, `docker run --privileged`, switches almost all of these protections off. If you understand capabilities and seccomp you can run services with the least privilege that still works.

Core concepts

Concept	Meaning
capability	One slice of root power, e.g. `CAP_NET_BIND_SERVICE`
permitted/effective set	Capabilities a process may use / is using now
bounding set	Upper limit that children can never exceed
no_new_privs	Flag preventing gaining privilege through exec (setuid)
seccomp	Kernel filter: allow, deny, or kill on specific syscalls
seccomp profile	JSON list of allowed syscalls, used by Docker/systemd
LSM	AppArmor or SELinux: path/label based mandatory access control

Common capabilities:

Capability	Allows
CAP_NET_BIND_SERVICE	Bind ports below 1024
CAP_NET_ADMIN	Change interfaces, routes, firewall
CAP_NET_RAW	Raw sockets (ping, packet capture)
CAP_SYS_ADMIN	A huge catch-all (mount, namespaces); avoid
CAP_CHOWN	Change file ownership
CAP_SYS_PTRACE	Trace other processes
CAP_DAC_OVERRIDE	Bypass file permission checks

```
process wants to do X
       |
  [seccomp filter]  -- syscall allowed? --no--> EPERM / kill
       |yes
  [capability check] -- has the cap?   --no--> EPERM
       |yes
  [LSM: AppArmor/SELinux] -- policy ok? --no--> denied
       |yes
     done
```

`--privileged` in Docker grants all capabilities, disables the default seccomp profile, exposes host devices, and relaxes the LSM. Treat it as "root on the host". The safer pattern is drop everything, add back the one you need.

Worked examples

See your own shell's capability sets:

```
grep Cap /proc/$$/status
capsh --decode=0000000000000000      # decode a hex mask (libcap tools)
getpcaps $$
```

Give a binary just one capability instead of setuid root:

```
sudo setcap 'cap_net_bind_service=+ep' ./myserver
getcap ./myserver
./myserver --port 80          # works as a normal user
sudo setcap -r ./myserver     # remove
```

Find risky files with capabilities or setuid bits:

```
getcap -r / 2>/dev/null
find / -xdev -perm -4000 -type f 2>/dev/null
```

Docker: least privilege run.

```
docker run --rm -it \
  --cap-drop ALL --cap-add NET_BIND_SERVICE \
  --read-only --security-opt no-new-privileges \
  --user 1000:1000 nginx:alpine
```

Compare with the one to avoid:

```
docker run --rm -it --privileged ubuntu:24.04 bash   # avoid in production
```

Check that a container is restricted:

```
docker run --rm alpine sh -c 'grep Cap /proc/self/status'
docker run --rm --privileged alpine sh -c 'grep CapEff /proc/self/status'
```

The privileged one shows a mask with nearly all bits set.

systemd hardening of a service (no Docker needed):

```
[Service]
User=app
NoNewPrivileges=yes
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
SystemCallFilter=@system-service
ProtectSystem=strict
PrivateTmp=yes
```

Audit the result:

```
systemd-analyze security myapp.service
```

Seccomp check from inside a container:

```
docker run --rm alpine sh -c 'grep Seccomp /proc/self/status'
# Seccomp: 2   (filter mode active)
```

Mini-lab

1. Print `CapEff` for your shell and for a non-privileged container. Compare.
2. Run a container with `--cap-drop ALL` and try `ping -c1 127.0.0.1`. Observe the failure, then add `--cap-add NET_RAW` and retry.
3. Use `setcap` to let a copy of a tiny Python HTTP server bind port 80 as a normal user (or explain why a container is simpler).
4. Run `systemd-analyze security` on an existing service and read the top three exposure findings.
5. Write down the exact Docker flag you would never put in a production run command. That is the exercise.

Decision guide / failure modes

Need	Choose
Bind port < 1024 without root	`CAP_NET_BIND_SERVICE` or use a high port + proxy
Run a debugger in a container	add `SYS_PTRACE` temporarily, not privileged
Network tooling (tcpdump)	`NET_ADMIN`/`NET_RAW` only
Need full device access	pass specific `--device`, not `--privileged`
Service on the host	systemd sandboxing options

Failure modes:

- "It works with –privileged" means you have not found the missing capability. Use `strace`, `dmesg`, and audit logs to find what was denied.
- Mounting `/var/run/docker.sock` into a container is equal to root on the host.
- Disabling seccomp (`seccomp=unconfined`) to fix one error removes all syscall filtering; write a small custom profile instead.
- Capabilities do not replace file permissions, users, or network policy; they are one layer.
- Ambient and file capabilities are lost across some exec paths and do not work with `no_new_privs` combinations you did not expect. Test the real unit.

Common mistakes

- Running containers –privileged by default
- Granting CAP_SYS_ADMIN casually
- Ignoring root in containers as "fine"

Exercises

Work in `exercises/32-capabilities-seccomp/`. Read that folder's `README.md` fully before coding.

```
cd exercises/32-capabilities-seccomp
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `avoid_docker_flag` in `exercise.sh` so it prints the single Docker run flag that grants a container nearly all host privileges and disables most sandboxing. Print the long option exactly as typed on the command line, including both leading dashes. When the test passes, write one paragraph in your notes explaining what safer pair of flags (drop and add) you would use instead and why the flag you printed is dangerous.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/32-capabilities-seccomp/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is a capability?
2. Why avoid –privileged?
3. seccomp filters what?

Further reading

- https://man7.org/linux/man-pages/man7/capabilities.7.html